AZURE DEVOPS

Automating the Software Delivery Lifecycle





THE PLATFORM

One platform for planning, building, testing, and delivering software

Microsoft platform bringing together developers, testers, operations, and release teams.

- Source control, work tracking, CI/CD, package management, and testing
- Agile and DevOps practices
- Integrates with GitHub, cloud platforms, containers, and Kubernetes

Five services that work together

Azure Boards

Work items, backlogs, sprints, tasks, project tracking

Azure Repos

Git repositories, branches, commits, pull requests

Azure Pipelines

Build, test, and deployment automation

Azure Test Plans

Manual testing, exploratory testing, test management

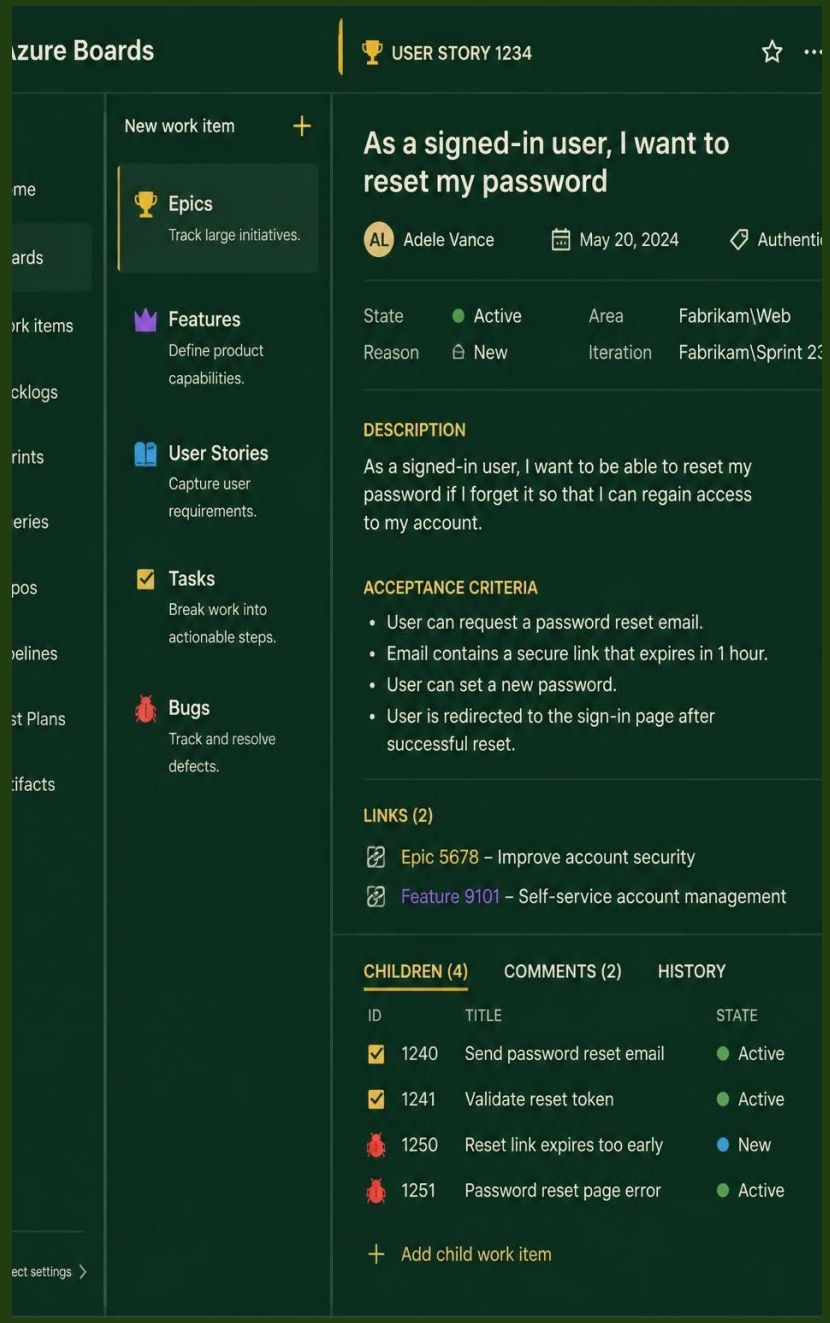
Azure Artifacts

Hosting and sharing packages through feeds

Azure Boards turns plans into trackable work

- Manage Epics, Features, User Stories, Tasks, and Bugs
- Organize backlogs and sprint boards; assign team members
- Track progress with Kanban boards and sprint dashboards
- Link code changes and pull requests to work items

A user story describes a new API feature; tasks break down the work; a bug records a defect found in testing.



Git and Azure Repos make review part of the workflow

Git provides distributed version control; Azure Repos hosts the team's repositories.

01	02	03	04	05	06
Developer	Feature Branch	Commit and Push	Pull Request	Review	Merge

Branches isolate feature development and fixes. Commits are pushed to the remote.

Pull requests drive review and controlled merging through branch policies, reviewer requirements, and build validation.

Azure Pipelines automates CI/CD

Continuous Integration and Continuous Delivery/Deployment, defined in YAML or the visual designer, depending on pipeline type.



Triggers start runs; agents execute jobs; stages organize build, test, and deploy.

Variables and service connections support configuration and integration.

YAML pipelines are versioned with your code

Stages are major phases such as Build, Test, Deploy.

Jobs are units of work executed on agents. Steps are scripts or tasks within a job.

Microsoft-hosted agents are managed by Microsoft; self-hosted agents are managed by the organization.

```yaml
trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- script: python --version
  displayName: 'Check Python'

- script: |
    pip install pytest requests
    pytest -v
  displayName: 'Run tests'
```

Catch defects early and keep unvalidated changes out

Automated tests

Run unit and integration tests in pipelines.

Pull request policies

Validate changes before merging.

Secrets and access

Never hardcode secrets in YAML. Use service connections for external services and least-privilege access to repositories, pipelines, environments, and resources.

Failure review

Review pipeline logs and test results to troubleshoot failures.

Pipeline artifacts are run outputs; Azure Artifacts is the package service

Pipeline artifacts

Binaries, JSON/CSV reports, deployment bundles, and build outputs

Consumed by later stages

Azure Artifacts

Hosts and shares NuGet, npm, Python, and Maven packages

Subject to feed configuration



Versioning ties a package or artifact to a release.

Environments and approvals govern every deployment

Development → Testing → Staging → Production

- Deployment jobs and environment resources model targets
- Approvals and checks control progression
- Environment-specific variables and service connections
- Validate health and connectivity after deployment
- Coordinate DNS, routing, firewall rules, and access with infrastructure or network teams

Deploy to dev, pause for approval, proceed once checks are satisfied.

The end-to-end workflow



A repeatable, traceable, and controlled software delivery process.

Best practices

Keep pipeline definitions in source control	Use feature branches and pull requests
Automate tests and validation	Promote the same tested artifact between environments where practical
Separate development and production permissions	Protect secrets and use least privilege
Document rollback, recovery, and release ownership	